Lagom: Unleashing the Power of Communication and Computation Overlapping for Distributed LLM Training

Guanbin Xu[†], ZhenGuo Xu[†], Yuzhe Li[†],
Youhui Bai[†], Ping Gong[†], Chaoyi Ruan[‡]
Cheng Li[†,°]
[†]University of Science and Technology of China ° Anhui Province Key Laboratory of Biomedical Imaging and Intelligent Processing,
Institute of Artificial Intelligence, Hefei Comprehensive National Science Center, China
[‡]National University of Singapore, Singapore

arXiv:2602.20656v1 [cs.DC] 24 Feb 2026

Abstract

Overlapping communication with computation is crucial for distributed large-model training, yet optimizing it—especially when computation becomes the bottleneck—remains challenging. We present Lagom, a system that co-tunes communication parameters to balance resource usage between computation and communication. By introducing a unified cost model and a priority-based search algorithm, Lagom reduces optimization complexity from exponential to linear. Evaluations on high- and low-bandwidth GPU clusters show that Lagom achieves 1.07-1.33× and 1.03-1.27× speedup over NCCL and AutoCCL across diverse models and parallelizations.

1 Introduction

The increasing scale of Deep Neural Networks (DNNs) is driving the need for distributed training, which is often limited by communication bottlenecks. To this end, a common optimization is to overlap communication with computation [2, 3, 19]. However, achieving optimal performance of overlap remains challenging, as communication operations contend with computation kernels for limited GPU resources. In practice, mature overlap strategies in industry often hide most communication latency [10], shifting the primary bottleneck to computation. Yet, even in such computation-bottlenecked scenarios, communication still interferes with computation, leaving overlooked opportunities for further optimization.

Existing approaches to managing contention have notable limitations. Liger [5, 30] mitigates contention by statically limiting communication's GPU resources, but its fixed allocation lacks adaptability. Libra [13] models overlap cost for GPU thread selection in data parallelism, yet it ignores key parameters (*e.g.* chunk size and number of channels) that we find have a greater impact on computation (Sec. 3.2). AutoCCL [29], the state-of-the-art communication tuner, addresses parameter tuning and is effective when communication is the bottleneck. While these approaches address specific aspects of overlap, they fail to holistically manage the dual-bottleneck scenario, especially when computation becomes the bottleneck where our analysis reveals that communication contention still degrades the performance of the bottlenecked computation by up to 35%.

Our analysis reveals that by strategically allocating fewer resources to communication, computation can utilize more resources, thereby accelerating the entire overlap. However, achieving this efficient overlap through parameter tuning is nontrivial, facing key challenges: (1) **Contention Complexity**: The relationship between the vast tunable parameter space and resource contention during the overlap of communication and computation operators—which must share multiple GPU resources—remains an open and poorly understood research problem. (2) **Exponential Search Space**: To prevent deadlock, collective communications are serialized, creating a chain of temporal dependencies. Consequently, tuning parameters for one communication (e.g., Comm1) affects not only its own execution but also cascades to delay dependent computations and other communications, as visualized in Fig. 1. This interdependence makes the joint optimization space grow exponentially with the number of communications—i.e., exceeding $1,000,000^N$ for N communications. An exhaustive search is infeasible, as it would require hundreds of GPU hours even for moderate N.

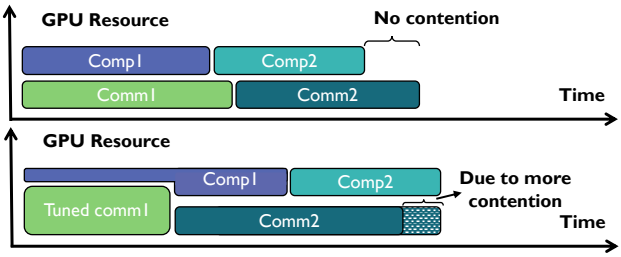


Figure 1: Side effects from tuning a single communication. Top: baseline execution. Bottom: after tuning Comm1, increased resource contention delays Comp2.

This paper presents Lagom, a novel system for tuning collective communication parameters. By introducing an overlap performance model and a resource-efficiency search heuristic, Lagom efficiently identifies near-optimal configurations for overlapping communication and computation. The main contributions are:

(1) Unified Overlap Performance Model: We formalize a unified cost model for computation-communication overlap that captures performance in both computation-bound and communication-bound regimes, establishing a foundational basis for optimization.

(2) Contention Analysis and Modeling: We analyze and categorize contention into two main types—SM competition and global resource contention—and model their relationship with tunable communication parameters.

Conference'17, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

(3) Tuning Algorithm and Evaluation: We introduce a fast search algorithm based on model-guided heuristics that reduces the optimization complexity and achieves 1.07-1.33× speedup across diverse models and parallelisms. The code for `Lagom` will be open-sourced soon.

2 Background and Motivation

2.1 Parallelism and Overlapping

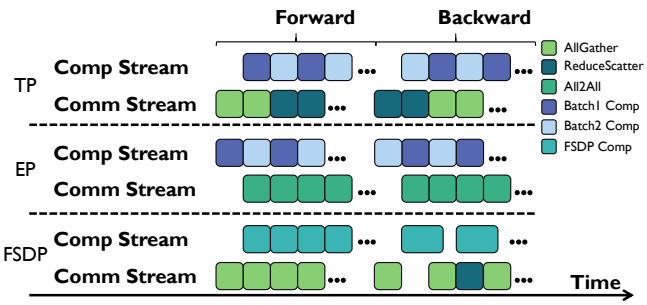


Figure 2: Overlaps of Different Parallelism

As data and model scales continue to grow, distributed training has become essential [6, 26]. Modern systems adopt various parallelisms that exploit communication–computation overlap as Fig. 2. The main forms include:

Tensor Parallelism (TP) [23] partitions large operators across devices to ease memory pressure, but incurs synchronization overhead (e.g., `AllReduce`). Domino [27] mitigates this via batch-level pipelining, though perfect overlap remains difficult.

Expert Parallelism (EP) [22] distributes experts across devices and relies on frequent `AlltoAll`. While efforts like the dual batch method aim to overlap communication with computation by resource partitioning [12], the cluster-specific resource allocation and AlltoAll-only implementation limit general applicability.

Fully Sharded Data Parallelism (FSDP) ([20, 31]) exhibits a more intricate overlap pattern, where each layer's computation interleaves with multiple communication phases (`AllGather` and `ReduceScatter`) across layers. Such multi-stage coupling makes concurrent optimization between computation and communication highly complex, often hindering ideal overlap.

These parallelisms are often combined with data parallelism [11, 21, 28] and pipeline parallelism [7, 15] to support even larger-scale training [9, 10, 16, 32].

2.2 Operators in Overlap and Parameter Tuning

Overlap involves over 1,500 computational operators [31], primarily implemented in closed-source libraries like cuBLAS [17] and cuDNN [18], making direct modification infeasible. In contrast, collective communications benefit from optimized open-source libraries like NCCL [1]. AutoCCL [29] identified six key parameters (Algorithm, Protocol, Transport, Number of Channels, Number of Threads, Chunk Size) that govern communication performance. They make a key observation that the configuration space can be addressed using a divide-and-conquer strategy. It first partitions the space into subspaces based on implementation-related parameters (Algorithm, Protocol, Transport), then models resource-related parameters (Number of Channels, Number of Threads, Chunk Size) within each subspace. Crucially, instead of explicitly modeling complex interference patterns, AutoCCL employs online sampling with real-time performance feedback to efficiently search resource-related parameters in each subspace. **We extend this finding by showing that these parameters also regulate GPU resource allocation during overlap, thereby critically affecting communication-computation contention.**

2.3 Challenges

However, the interaction between communication and computation is inherently difficult. The dynamic nature of multi-communication tuning expands the search space exponentially, making joint optimization challenging. These challenges mainly arise from: (1) **Complex Contention Between Communication and Computation.** Communication operations share core GPU resources such as Streaming Multiprocessors (SMs), global memory bandwidth, and L2 cache with computations. The degree of contention varies with communication type, message type, concurrency, and hardware design, making performance modeling highly nonlinear. (2) **Exponential Growth of the Search Space for Multi-Communications.** In overlapping scenarios, multiple communications interfere with each other and with computation. For instance, tuning the parameters of one communication operation may change its completion time, which, in turn, affects the overlap between subsequent communications and computation, thereby altering the overall system performance. With n communication operations each having k configuration options, the joint search space grows as k^n, which calls for an effective reduction in dimensionality.

3 Design

3.1 Problem Definition

Table 1: Key notations used in the cost model.

Notation	Description
X, Y, Z	Communication, computation, and total time.
s_j	j-th communication config (A, P, T, NC, NT, C).
TB_i	i-th computation's threadblocks.
λ	Total number of SMs.
D_i	Data size per thread block in the i-th computation.
$\overline{B}$	Peak global memory bandwidth.
V_j	Global bandwidth of the j-th communication.

$$\min_{\{s_1,\ldots s_N\}} Z = \max(Y, X) = \max\left(\sum_{i=1}^{M} y_i,\ \sum_{j=1}^{N} x_j^{s_j}\right) \quad (1)$$

To address these challenges, we now introduce a unified cost model and analyze its behavior across different bottleneck scenarios in computation-communication overlap. For an overlap, let it contain M computation operators and N communication operators. We denote the execution time of the i-th computation operator as y_i, and the execution time of the j-th communication operator under configuration s_j as $x_j^{s_j}$. With computations and communications serialized on separate CUDA streams, the total computation time is

$Y = \sum_{i=1}^{M} y_i$, the total communication time is $X = \sum_{j=1}^{N} x_j^{s_j}$, and the overall makespan is given by $Z = max(X, Y)$. As shown in Equation (1), the optimization objective is to minimize the overall makespan Z by selecting optimal configurations s_j for all communication operators. We now analyze the two bottleneck scenarios separately.

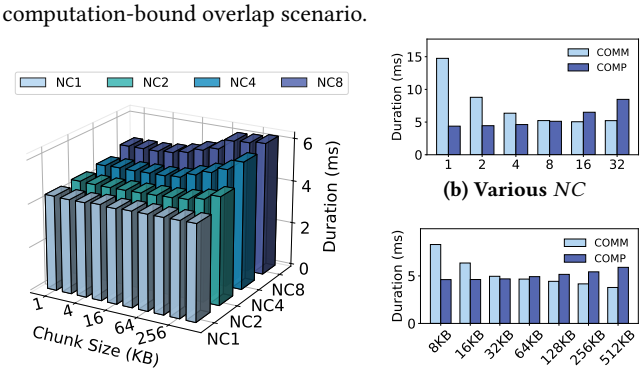
$$\min_{\{s_1,\ldots s_N\}} Z = \min_{\{s_1\ldots s_N\}} \sum_{j=1}^{N} x_j^{s_j} = \sum_{j=1}^{N} \min_{s_j} x_j^{s_j} \tag{2}$$

Communication Bottleneck. First, in the communication-bound case described by Equation 2, the overlap makespan simplifies to the cumulative time of all communications. The key challenges are: (1) modeling how configuration s_j affects communication time $x_j^{s_j}$, and (2) capturing the impact of computational contention on communication. AutoCCL [29] addresses this by employing a subspace coordinate descent method for search, and leverages online sampling with real-time feedback to avoid explicit modeling of the contention.

$$\min_{\{s_1\ldots s_N\}} Z = \sum_{i=1}^{M} y_i \tag{3}$$

Computation Bottleneck. Symmetrically, Equation 3 describes the computation-bound scenario, where the overlap makespan is determined by the sum of computation operator times. Two key challenges arise in this case: (1) Communication parameters influence computational performance by governing how communications utilize shared resources (further discussed in Sec. 3.2); (2) Due to the serialized execution of communications, tuning an earlier communication changes the start time and overlap window of subsequent operators, creating a cascade effect on the computation. This requires considering the entire configuration set $s_1, s_2, \ldots, s_N$ jointly. With the configuration space per communication exceeding $r = 10^6$ options [29], the joint search space becomes $O(r^N)$ for N communication operators, which is computationally intractable for practical values of N.

We now analyze how to tune communication parameters in the computation-bound overlap scenario.



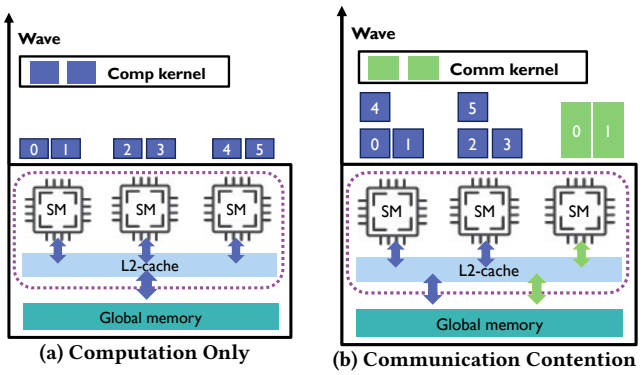
(a) Various NC and C (b) Various NC (c) Various C

Figure 3: Duration of FFN when AllReduce(32MB) with various NC and C (NT=128) on 8 × A40pcie

3.2 Contention Modeling

Building on AutoCCL [29], we adopt a divide-and-conquer strategy based on implementation-related parameters (Algorithm, Protocol, Transport) before analyzing the effect of resource-related parameters. Our key finding is that while different communication configurations can achieve similar communication performance, they exhibit significant variation in how much they slow down computation. As shown in Fig. 3, when using NC=16 versus NC=32, communication performance remains nearly identical, but computation time differs by 30.2%.

We conducted separate tests on NC, NT, and C, and found that NC and C have a significant impact on computation time, while the effect of NT is negligible. Fig. 3a shows the overall impact of different NC and C configurations on computation time, measured by concurrently running a 32 MB AllReduce and an FFN operator on 8 NVIDIA A40 GPUs with PCIe interconnect. The results indicate that increasing either NC or C progressively prolongs computation time. To isolate individual factors, Fig. 3b shows how communication and computation times vary as NC increases while $C = 16KB$. Similarly, Fig. 3c shows the effect of increasing C when $NC = 4$. The figures confirm that while larger NC and C values initially reduce communication time (with diminishing returns or even slight increases at larger values), they also significantly slow down computation. These findings were reproduced consistently on another cluster.



(a) Computation Only (b) Communication Contention

Figure 4: SM and global resource contention.

Cost Model We characterize communication-computation contention along two dimensions: SM competition and global resource competition. As illustrated in Fig. 4, SM competition arises from shared processor occupancy, while global resource contention stems from bandwidth saturation. **SM Competition.** Communication operators occupy a portion of SMs, reducing the resources available for computation and increasing the number of computation waves. The assignment of threadblocks, whether for computation or communication, to an SM is constrained by the availability of threads, shared memory, and registers. Due to this multi-constraint bottleneck, NT has a negligible impact on SM competition. NC is the dominant factor governing SM occupancy. **Global Resource Competition.** Communication also contends for shared global resources such as memory bandwidth and cache. Each communication operator launches NC threadblocks that read/write data at a granularity of C. Memory transactions from different threads

within a threadblock are coalesced, making NT's impact on global contention relatively minor. In contrast, larger values of NC and C generate more intensive contention due to increased concurrent data movement.

$$y_i = \sum_{j=l}^{k} f_{ij} \times g_{ij} \tag{4}$$

$$g_{ij} = \lceil \frac{\mu_i}{(\lambda - NC_j) \times TB_i} \rceil \tag{5}$$

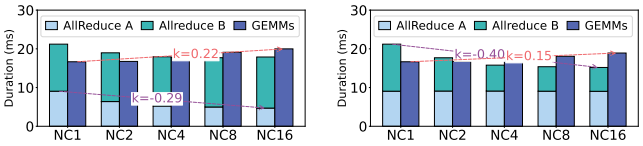
$$f_{ij} = \theta_{ij} + \frac{(\lambda - NC_j) \times TB_i \times D_i}{\overline{B} - V(NC_j, C_j)} \tag{6}$$

Based on these observations, we build a cost model of the computation i overlapped with communication $l-k$. Eq. (4) decomposes computation time y_i into wave count g_{ij} and per-wave latency f_{ij} that is overlapped with the j-th communication. Eq.(5) shows the number of computation waves g_{ij} is determined by dividing the total required thread blocks μ_i by the number of available threadblocks per SM after allocating NC_j SMs to communication. Meanwhile, Eq. (6) shows f_{ij} consists of computation time θ_{ij} and data transfer time, where the latter depends on the data volume D_i and the effective bandwidth reduced by global resource contention $V(NC_j, C_j)$. Notation definitions are summarized in Table 1. A higher NC occupies more SMs, reducing the resources available for computation and consequently increasing the number of waves g_i, while also imposing additional demand on bandwidth. A larger C, in turn, amplifies contention for global bandwidth, thereby extending the per-wave execution time f_i. Fig. 3 empirically validates our model: increasing NC/C beyond optimal points (e.g., NC>8) drastically prolongs computation time despite marginal communication gains.

3.3 Priority Metric

When multiple communication and computation tasks execute concurrently, the central challenge lies in efficiently identifying near-optimal configurations from a combinatorial space that scales as $O(1,000,000^n)$. Brute-force searching this space is infeasible, necessitating the use of heuristic methods to find high-quality solutions within practical time constraints. A naive strategy is to optimize communications sequentially. However, this approach fails to account for critical interdependencies: tuning one communication alters its execution timeline, which cascades to affect both the communication and computation phases of subsequent tasks. These temporal and resource-contentions introduce nonlinear interactions that render sequential optimization highly suboptimal. To illustrate this, an experiment was conducted on A40 GPUs with 2 AllReduce and 7 MatMul executed concurrently. We adjust NC of one communication at a time to analyze how it impacts overall performance.

As shown in Fig. 5, increasing NC from 1 to 16 affects the two communications differently. The slope of each arrow indicates the rate of change in computation and communication times. Optimizing Communication B achieves a larger reduction in communication time with only a slight increase in computation time, resulting in a shorter total execution time. In contrast, tuning Communication A provides smaller benefits. These results reveal that optimizing different communications yields distinct trade-offs between communication gains and computational overhead. The overall performance improvement thus depends on selectively prioritizing the communications that provide higher benefits. This motivates the introduction of the metric H, which quantifies such trade-offs across concurrent communications.



(a) tuning AllReduce A only **(b) tuning AllReduce B only**

Figure 5: Cost differences when tuning different communications in multiple communication scenarios

Metric H. We define a priority metric H_j for the j-th communication as Equation (7). Y and Y' denote the computation times before and after tuning, while $x_j^{s_j}$ and $x_j^{s'_j}$ represent the corresponding communication times for the j-th communication. Where Y and Y' represent the computation time before and after optimization, respectively, and $x_j^{s_j}$ and $x_j^{s'_j}$ represent the communication time corresponding to the two configurations s_j and s'_j for the j-th communication.

$$H_j = \frac{Y' - Y}{x_j^{s_j} - x_j^{s'_j}} \tag{7}$$

Hence, a smaller numerator indicates a lower computational overhead, and a larger denominator implies greater communication improvement. The metric H_j measures the tuning efficiency, expressed as the communication improvement achieved per unit of added computation cost. A negative denominator implies that the communication is already optimal.

Noted: in multi-computation scenarios, tuning one communication affects multiple overlapped computations, making explicit modeling of each interference infeasible. Since H_j inherently reflects the overall impact of the tuning on all computations, the analysis can be simplified by considering only the aggregate computation time Y, significantly reducing complexity.

3.4 Search Method

In an overlap, the optimal communication configuration satisfies one of the following conditions: (1) All communications use minimal resources, and their total time cost is less than the computation time. (2) Communications are fully optimized but still take longer than computation, and further increasing communication resources prolongs both communication and computation time. (3) Increasing communication resources reduces communication time but increases computation time until the two are equal—beyond this point, further resource allocation for communication continues to speed up communication while slowing computation even more, degrading overall performance.

To efficiently identify the optimal configuration that satisfies the boundary conditions, we designed Algorithm 1 and Algorithm 2. **Algorithm 1** iteratively optimizes communications within an overlap group. In each iteration, it selects the communication with the

Algorithm 1: Cost-Effectiveness Algorithm

Input : Communication config set $S = \{s_1, s_2, ..., s_n\}$
Output : fine-tuned communication config S'

1 $S' \leftarrow \{\}$
2 Initialize all H to 0.01
3 **while** $\exists s \in S$ *and* $s.done = False$ **do**
4 $\quad s_j \leftarrow \arg\min_{s_i \in M, s_i.done=\text{False}} H_i$
5 $\quad s'_j \leftarrow \text{ResourceEfficientTuning}(s_j)$
6 $\quad$ **if** $s'_j.done$ **then**
7 $\quad\quad S'.\text{append}(s_j)$
8 $\quad$ **else**
9 $\quad\quad H_j \leftarrow \frac{Y' - Y}{x_j^{s_j} - x_j^{s'_j}}$
10 $\quad$ **end**
11 **end**
12 **return** S'

Algorithm 2: Resource-Efficient Tuning Algorithm

Input : Communication config s'_j

1 **if** s'_j *is not initialized* **then**
2 $\quad s'_j.NC, s'_j.NT, s'_j.C \leftarrow NC_{min}, NT_{min}, C_{min}$
3 **end**
4 $x_j^{s'_j}, Y', X' \leftarrow \text{ProfileTime}(s'_j)$
5 **if** $x_j^{s'_j} - x_j^{s_j} > 0$ *or* $X' < Y'$ **then**
6 $\quad s'_j.done \leftarrow \text{True}$
7 **else**
8 $\quad lr \leftarrow \frac{x_j^{s'_j} - x_j^{s_j}}{x_j^{s'_j}}$
9 $\quad s'_j.NC \leftarrow s'_j.NC + lr$
10 $\quad s'_j.NT \leftarrow s'_j.NT + lr$
11 $\quad s'_j.C \leftarrow s'_j.C + lr$
12 **end**
13 **return** s'_j

smallest H from the unfinished set (line 4), then invokes Algorithm 2 to determine the next configuration for the selected communication (line 5). If the communication is marked as completed, its configuration is added to the output set (line 7). Otherwise, the algorithm updates the corresponding H based on the latest measurements (line 9). The process repeats until all communications have been tuned.

Algorithm 2 generates the next configuration for a given communication. If the communication has not been initialized, NC, NT, and C are set to their minimum values (lines 1-3). Otherwise, the algorithm checks whether the current configuration satisfies either of the following termination conditions (line 5): (1) the communication itself has reached its optimal performance, or (2) the communication time is less than the computation time. If neither condition is met, the algorithm increases the values of NC, NT, and C by a learning rate lr computed from the relative improvement in communication time (lines 8–11).

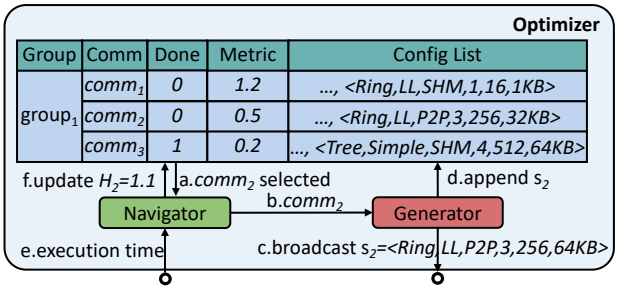


Figure 6: Workflow of tuning multi-comms in one overlap

We integrate the proposed algorithms into AutoCCL's divide-and-conquer framework to tune resource-related parameters. The overall workflow is illustrated in Fig. 6, using an overlap with 3 communication as an example. The algorithm begins by selecting the communication with the smallest H among those not yet tuned (*e.g.*, $comm_2$ in step a). It then increases the NC, NT, and C for this communication appropriately. The new configuration s_2 is added to $comm_2$'s configuration list (step d) and broadcast to all ranks for execution (step c). After measuring the execution time (step e), the H for $comm_2$ is updated (step f). This process repeats iteratively until all communications have been tuned. This approach identifies minimal resource-related parameter that balance communication and computation performance.

4 Evaluation

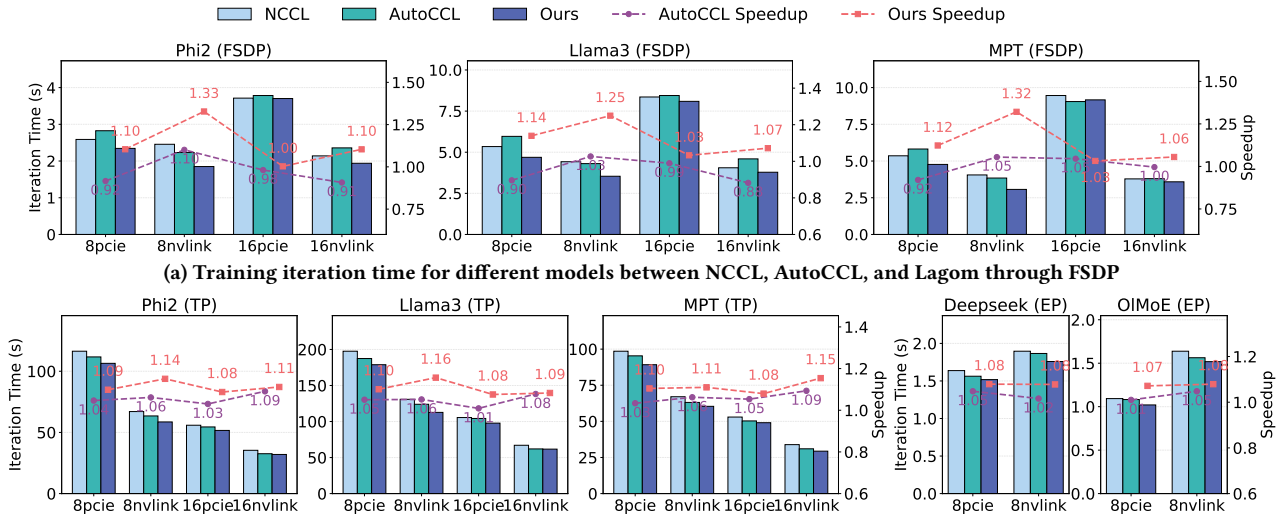
4.1 Experimental Setup

Table 2: DNN model statistics for different parallelism (8-16 A40 GPUs). MBS/GBS denotes Micro/Global Batch Size.

Model	MBS	GBS	TP	DP	EP	FSDP
Phi-2-2B	2	$2 \times [8, 16]$	–	–	–	8,16
Llama-3-8B	1	$2 \times [8, 16]$	–	–	–	8,16
MPT-7B	1	$2 \times [8, 16]$	–	–	–	8,16
Phi-2-2B	8	512	8	1,2	1	–
Llama-3-8B	4	256	8	1,2	1	–
MPT-7B	2	256	8	1,2	1	–
DeepSeek-MoE-16B	2	16	1	1	8	–
OLMoE-1B-7B	2	16	1	1	8	–

Workload and Software As shown in Table 2, we evaluate our method on a diverse set of large-scale models, including dense models (Phi-2B [8], Llama-3-8B [25], and MPT-7B [24]) as well as Mixture-of-Experts (MoE) models (DeepSeek-Moe (16B) [4] and OLMoE-1B-7B [14]). Our implementation is built upon Megatron-LM [23] and Pytorch 2.3.0, with Data Parallelism (DP), Tensor Parallelism (with DeepSpeed-Domino [27]), Expert Parallelism (with two batch overlapping [30]), and FSDP. Across all experiments, the micro-batch size was set to the maximum value that could fit within GPU memory, while the global batch size followed widely-adopted industry standards [29].

Hardware Infrastructure. To evaluate the generality of our approach, we use two distinct hardware clusters. **Cluster A** consists of 2 nodes, each with 8 NVIDIA A40 GPUs, connected internally



(a) Training iteration time for different models between NCCL, AutoCCL, and Lagom through FSDP

(b) Training iteration time for different models between NCCL, AutoCCL, and Lagom through TP/EP.

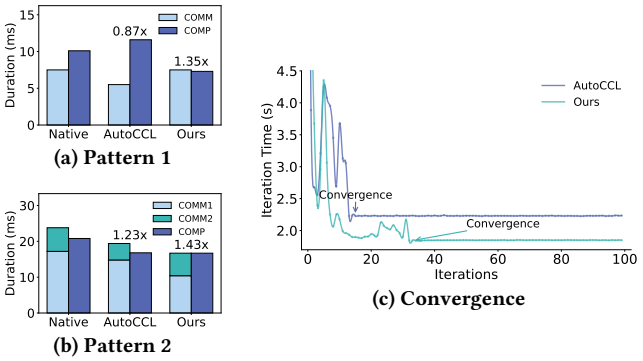
Figure 7: Overall comparison of iteration time across communication strategies in FSDP (a) and TP/EP (b)

via NVLink with full 400 Gbps connectivity and between nodes using 2×400 Gbps InfiniBand. **Cluster B** also has 2 nodes with 8 NVIDIA A40 GPUs per node, but uses PCIe 4.0 for intra-node connections and 100 Gbps InfiniBand for inter-node communication. The software environments are CUDA 12.8 with Driver 570.13.

Baseline We compare our approach against two baselines: NCCL v2.18.3-1 and AutoCCL. Our implementation is built on NCCL and AutoCCL, and can be integrated non-intrusively into modern DNN training frameworks.

4.2 End-to-end Performance

Fig. 7a presents `Lagom` consistently achieves 1.10-1.33× performance over NCCL across different clusters and models with FSDP by automatically identifying appropriate configurations that balance resource utilization. In contrast, AutoCCL's strategy of aggressively tuning communication parameters can lead to worse end-to-end performance than NCCL in computation-bound scenarios since excessive resource allocation to communication exacerbates the computational bottleneck, as predicted by our contention model. Furthermore, `Lagom` demonstrates greater performance gains on cluster A since NCCL defaults to larger NC values to exploit the available bandwidth when GPUs are connected via NVLink, which significantly increases contention and worsens computational bottlenecks. Fig. 7b shows `Lagom` achieves speedups of 1.08–1.16× over NCCL in tensor parallelism and 1.07–1.08× in expert parallelism, demonstrating its general applicability across different parallelism. While AutoCCL also achieves notable speedups of 1.03-1.09×, it consistently underperforms `Lagom` since AutoCCL primarily optimizes communication-bounded overlap scenarios, whereas `Lagom` effectively handles diverse bottleneck that occur throughout the training iteration. We provide a detailed breakdown of these scenarios in Section 4.3.



(a) Pattern 1

(b) Pattern 2

(c) Convergence

Figure 8: Phi2B model breakdown and convergence

4.3 Breakdown

We profile the Phi-2-2B model under FSDP on NVLink within a single node and identify two dominant overlap patterns (Fig. 8) covering approximately 90% of end-to-end training time. As shown in Fig. 8, we analyze the FSDP training of the Phi-2-2B model on a single node in Cluster A. In Pattern 1, which is computation-bound, NCCL uses the default configuration of NC=8 and C=2 MB for the `AllGather`. AutoCCL increases the NC to 61 to reduce communication time, but this introduces severe contention and degrades overall throughput to 0.87×. In contrast, `Lagom` adopts a configuration of NC=2 and C=684 KB, which slightly reduces communication speed but significantly alleviates contention, yielding an end-to-end speedup of 1.35×. Pattern 2 involves multiple communications. `Lagom` prioritizes the tuning of the `ReduceScatter` based on its H, adjusting its parameters from (NC=8, C=2 MB) to (NC=4, C=1366 KB), and achieves a 1.43× speedup.

4.4 Efficiency of Tuning

Fig. 8c validates that `Lagom` can find the optimal configuration with linear search complexity. In an overlap scenario involving two communication tasks, AutoCCL converges within 16 iterations while

Lagom converges in 33 iterations, showing a ratio of approximately 1:2, which aligns with the predicted linear complexity. The resulting convergence overhead is negligible compared to a full training run (over 1 million iterations [10, 23]), demonstrating the search efficiency and practical utility of Lagom.

5 Conclusion

In this paper, we present Lagom, a novel co-tuning method that automatically and efficiently optimizes communication parameters to speed up the performance of overlap. We formalize a cost model for overlap and comprehensively analyze its key challenges under different scenarios. To address the exponential search complexity, we introduce a cost-benefit metric and the tuning algorithm that reduces the complexity to linear. We implement Lagom on top of NCCL and evaluate it on different clusters. Results show that Lagom achieves 1.07-1.33× and 1.03-1.27× performance over both NCCL and AutoCCL across various large-model parallel training.

References

[1] 2025. NVIDIA Collective Communication Library (NCCL). https://github.com/nvidia/nccl. [Accessed 08-11-2025].

[2] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. 2024. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 178–191.

[3] Shenggan Cheng, Shengjie Lin, Lansong Diao, Hao Wu, Siyu Wang, Chang Si, Ziming Liu, Xuanlei Zhao, Jiangsu Du, Wei Lin, et al. 2025. Concerto: Automatic Communication Optimization and Scheduling for Large-Scale Deep Learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 198–213.

[4] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. arXiv:2401.06066 [cs.CL] https://arxiv.org/abs/2401.06066

[5] Jiangsu Du, Jinhui Wei, Jiazhi Jiang, Shenggan Cheng, Dan Huang, Zhiguang Chen, and Yutong Lu. 2024. Liger: Interleaving Intra-and Inter-Operator Parallelism for Distributed Large Model Inference. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 42–54.

[6] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. 2021. DAPPLE: A pipelined data parallel approach for training large models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 431–445.

[7] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).

[8] Alyssa Hughes. [n. d.]. Phi-2: The surprising power of small language models — microsoft.com. https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/. [Accessed 07-08-2025].

[9] Xianyan Jia, Le Jiang, Ang Wang, Jie Zhang, Xinyuan Li, Wencong Xiao, Yong Li, Zhen Zheng, Xiaoyong Liu, Wei Lin, et al. 2020. Whale: Scaling deep learning model training to the trillions. *arXiv preprint arXiv:2011.09208* (2020).

[10] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. 2024. MegaScale: Scaling large language model training to more than 10,000 GPUs. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 745–760.

[11] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668* (2020).

[12] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).

[13] Yunzhuo Liu, Bo Jiang, Shizhen Zhao, Tao Lin, Xinbing Wang, and Chenghu Zhou. 2023. Libra: Contention-Aware GPU Thread Allocation for Data Parallel Training in High Speed Networks. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 1–10.

[14] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, Noah A. Smith, Pang Wei Koh, Amanpreet Singh, and Hannaneh Hajishirzi. 2025. OLMoE: Open Mixture-of-Experts Language Models. arXiv:2409.02060 [cs.CL] https://arxiv.org/abs/2409.02060

[15] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 1–15.

[16] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.

[17] Nvidia. 2025. Basic Linear Algebra on NVIDIA GPUs. https://developer.nvidia.com/cublas. [Accessed 08-11-2025].

[18] Nvidia. 2025. CUDA Deep Neural Network. https://developer.nvidia.com/cudnn. [Accessed 08-11-2025].

[19] Yanghua Peng, Yibo Zhu, Yangrui Chen, Yixin Bao, Bairen Yi, Chang Lan, Chuan Wu, and Chuanxiong Guo. 2019. A generic communication scheduler for distributed dnn training acceleration. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 16–29.

[20] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.

[21] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, HyoukJoong Lee, Mingsheng Hong, Cliff Young, et al. 2018. Mesh-tensorflow: Deep learning for supercomputers. *Advances in neural information processing systems* 31 (2018).

[22] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).

[23] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[24] The MosaicML Research Team. 2023. Introducing MPT-7B: A New Standard for Open-Source, Commercially Usable LLMs. https://www.databricks.com/blog/mpt-7b [Accessed 07-08-2025].

[25] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[26] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Pat McCormick, Jamaludin Mohd-Yusof, et al. 2022. Unity: Accelerating DNN Training Through Joint Optimization of Algebraic Transformations and Parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 267–284.

[27] Guanhua Wang, Chengming Zhang, Zheyu Shen, Ang Li, and Olatunji Ruwase. 2024. Domino: Eliminating Communication in LLM Training via Generic Tensor Slicing and Overlapping. *arXiv preprint arXiv:2409.15241* (2024).

[28] Minjie Wang, Chien-chin Huang, and Jinyang Li. 2019. Supporting very large models using automatic dataflow graph partitioning. In *Proceedings of the Fourteenth EuroSys Conference 2019*. 1–17.

[29] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. 2025. {AutoCCL}: Automated Collective Communication Tuning for Accelerating Distributed and Parallel {DNN} Training. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 667–683.

[30] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. https://github.com/deepseek-ai/DeepEP.

[31] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277* (2023).

[32] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Joseph E Gonzalez, et al. 2022. Alpa: Automating Inter-and Intra-Operator Parallelism for Distributed Deep Learning. *arXiv preprint arXiv:2201.12023* (2022).